

POLITECNICO
MILANO 1863

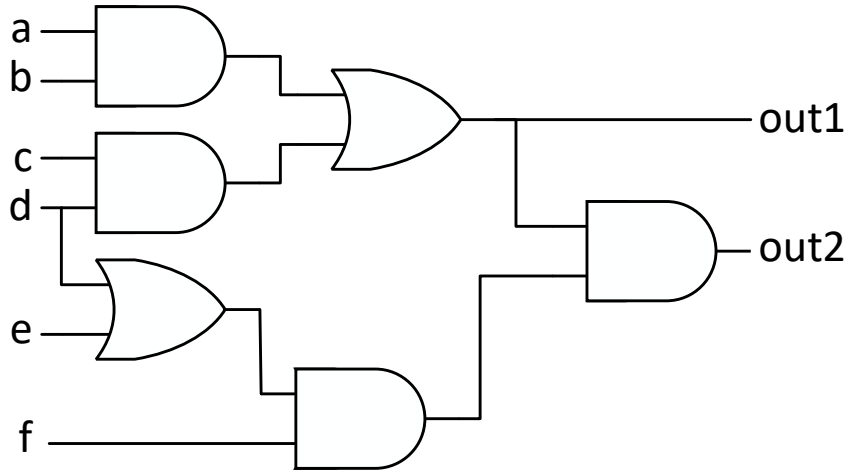
Reti Logiche

Sintesi di reti combinatorie a più livelli

Introduzione

Da sintesi a due livelli a sintesi multilivello

- Il **riuso di sottoparti di un circuito** permette notevoli riduzioni del costo di sistemi multi-uscita
  - La sintesi a due livelli sfrutta in modo limitato il riuso solo sui termini prodotto
- I sistemi digitali sono in genere **descritti a livello register-transfer** come uno schematico che interconnette varie unità funzionali
  - Tale schematico si adatta poco ad un modello di circuito a due livelli
- Si abbandona quindi il modello di riferimento a due livelli per considerare un modello più generale, detto **circuito multilivello**



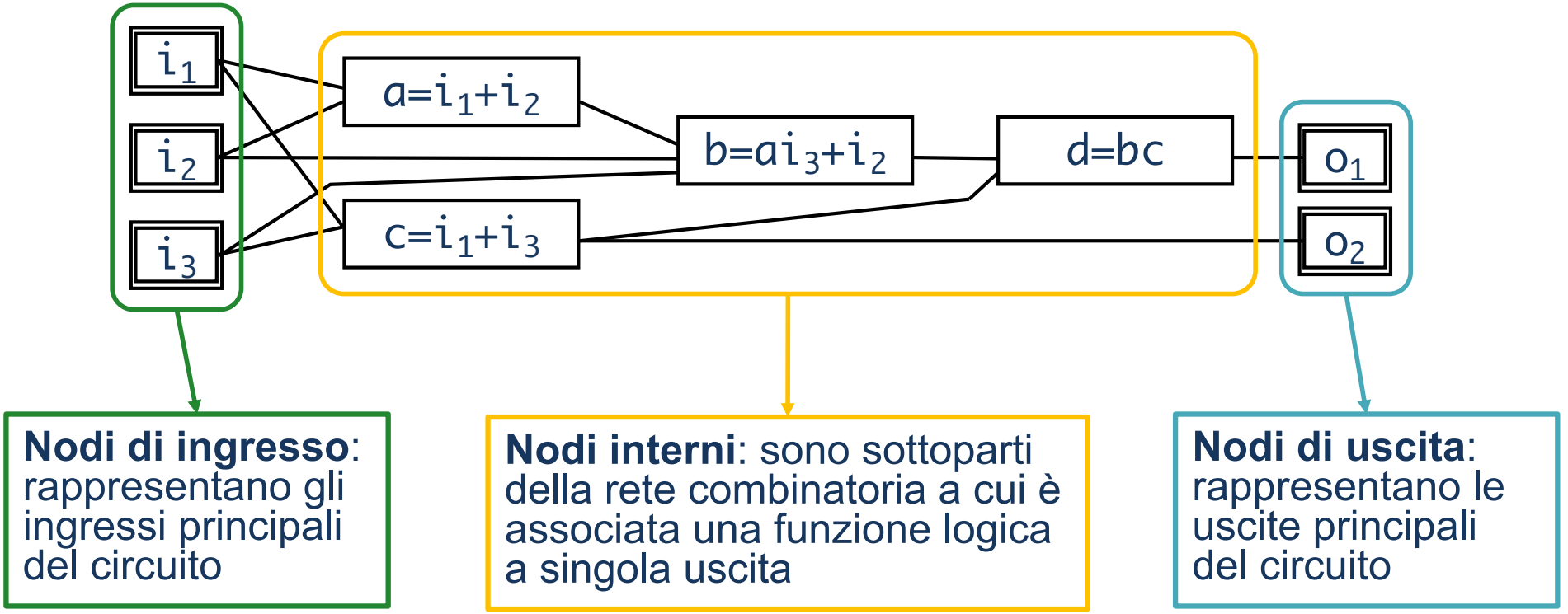
Implicazioni del modello multilivello

- Il rilassamento del vincolo dei due livelli porta ad un aumento considerevole del numero di possibili circuiti che realizzano una data funzione
  - La sintesi non può più esser affrontata mediante metodi esatti
  - Si adottano perciò **metodi euristici**

- Il processo di ottimizzazione multilivello può avere impatti contrastanti su area e latenza del circuito
  - Nella sintesi a due livelli l'ottimizzazione della complessità porta ad una minimizzazione dell'area e implicitamente anche della latenza
  - Nella sintesi a più livelli le due metriche non procedono nella stessa direzione. Possiamo quindi:
    - Minimizzare l'area (con vincolo sulla latenza)
    - Minimizzare la latenza (con vincolo sull'area)

Modello

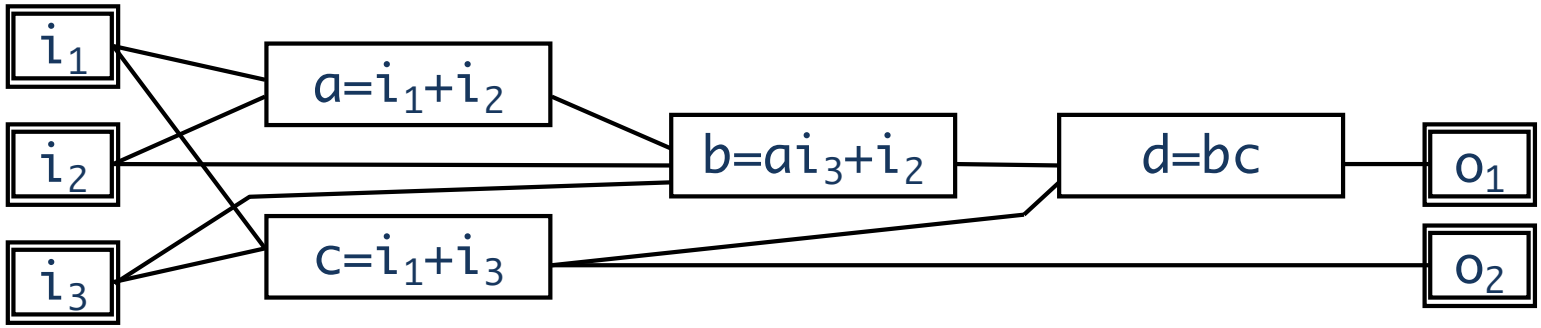
- La sintesi multilivello adotta una rappresentazione del circuito combinatorio basato su un grafo diretto aciclico



- Gli archi rappresentano le connessioni dal nodo che definisce una variabile al nodo che la usa

Modello

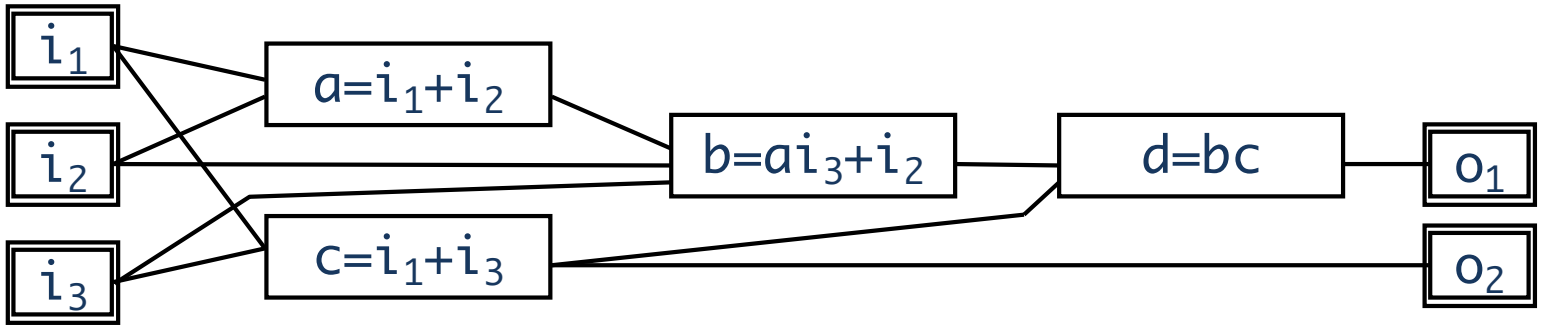
- La sintesi multilivello adotta una rappresentazione del circuito combinatorio basato su un grafo diretto aciclico



- Modello comportamentale/strutturale:
  - **Strutturale**: rappresenta le connessioni tra sottoparti
  - **Comportamentale**: ad ogni nodo è associata una **funzione logica** espressa mediante un'espressione **a due livelli con una sola uscita**

Modello

- La sintesi multilivello adotta una rappresentazione del circuito combinatorio basato su un grafo diretto aciclico



- Il grafo può esser riassunto con un elenco di equazioni:

$a=i_1+i_2$
$c=i_1+i_3$
$b=ai_3+i_2$
$d=bc$
$o1=d$
$o2=c$

Sintesi multilivello

- La sintesi di circuiti multilivello si basa sull'applicazione iterativa di una sequenza di **trasformazioni** per manipolare la rete

- Ciascun tipo di trasformazione è descritto mediante un algoritmo
  - Si basa sulle regole dell'algebra booleana per modificare la rete
  - **Preserva l'equivalenza logica del circuito ai morsetti**

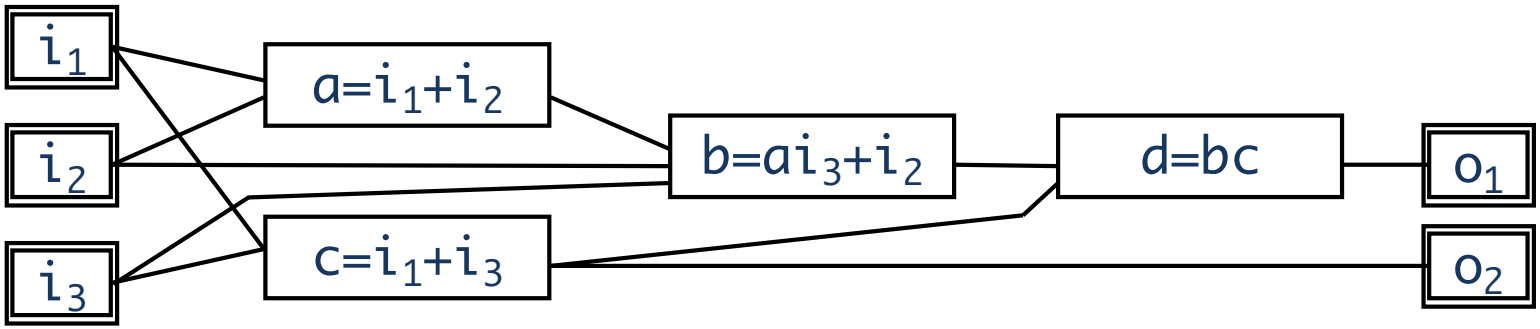
- La sequenza di trasformazioni da applicare non è prefissata
  - Differenti sequenze portano a risultati differenti

- La sintesi si ferma quando le trasformazioni non portano più ad alcun beneficio

Figure di merito

- La rete è valutata mediante due figure di merito:
  - **Area**: numero di letterali
  - **Latenza**: lunghezza del percorso critico dagli ingressi alle uscite assumendo che ciascun nodo abbia un ritardo unitario

- Esempio:



Area = 2 + 3 + 2 + 2 = 9
Latenza = 3 (il percorso critico va da i_1 a o_1)

Trasformazioni

- Esistono due classi di trasformazioni:
  - **Trasformazioni locali**: modificano l'espressione all'interno di un nodo lasciando invariata la topologia complessiva della rete
  - **Trasformazioni globali**: modificano sia l'espressione di uno o più nodi ed anche la topologia complessiva della rete



POLITECNICO
MILANO 1863

Reti Logiche

Sintesi di reti combinatorie a più livelli

Trasformazioni

Elenco delle operazioni

- **Valutazione del costo**
  - `cost`
- **Trasformazioni locali**
  - `simplify`
  - `factor`
- **Trasformazioni globali**
  - `eliminate`
  - `sweep`
  - `substitute`
  - `extract`
  - `decompose`

Cost

- `cost()` calcola il costo della rete in termini di **numero di letterali**
- Non è una trasformazione

- Esempio:

```
                    cost()
d = a + b + c         3 +
x = de + ef           4 +
y = df + e            3 =
                     ---
                      10
```

Simplify

Trasformazione locale

- `simplify(f)` semplifica mediante uno dei metodi di sintesi ottima studiati l'espressione del nodo `f` portandolo alla **forma minima a due livelli**

- Esempio: `simplify(f)`

`f = abcd + ab'cd + abcd'` Costo = 12

`f(a,b,c,d) = on-set(11, 14, 15)`

ab \ cd	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	1	1
10	0	0	1	0

Implicanti primi essenziali:
A = abc
B = acd

Soluzione:
`f = abc + abd` Costo = 6

Factor

Trasformazione locale

- `factor(f)` **fattorizza l'espressione** del nodo n espressa nella forma su due livelli ottenendo un'espressione su più livelli
- La fattorizzazione è ottenuta mediante un'euristica che individua quali letterali raccogliere ed in che ordine
- Algoritmo:
  1. Si individua il letterale che compare volte nell'espressione
  2. Si raccoglie a fattor comune tale letterale ottenendo due espressioni in OR:
     - Una come somma dei termini prodotto su cui è stato effettuato il raccoglimento, moltiplicato per il letterale raccolto
     - Una come somma dei termini prodotto rimanenti
  3. Si ripete ricorsivamente l'analisi sulle due sottoespressioni finché è possibile
  - Nel caso ci siano più letterali ad uguale frequenza massima si sceglie di dar priorità in base ad un ordine lessicografico decrescente o crescente

Factor

Trasformazione locale

➡ Esempio: `factor(f)` `f = acd + a'bc' + a'bd' + b'cd` Costo = 12

	a	a'	b	b'	c	c'	d	d'
acd	1	0	0	0	1	0	1	0
a'bc'	0	1	1	0	0	1	0	0
a'bd'	0	1	1	0	0	0	0	1
b'cd	0	0	0	1	1	0	1	0
	1	2	2	1	2	1	2	1

dc

	a	b'
a	1	0
b'	0	1
	1	1

b' + a

+

ba'

	a'	b	c'	d'
a'bc'	1	1	1	0
a'bd'	1	1	0	1
	2	2	1	1

	c'	b'
c'	1	0
d'	0	1
	1	1

d' + c'

+ 0

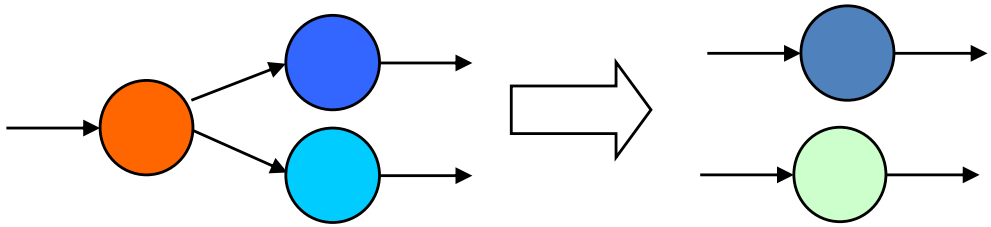
Risultato:

`f = dc(b' + a) + ba'(d' + c')` Costo = 8

Eliminate

Trasformazione globale

- `eliminate(f)` **elimina** il nodo n sostituendo in ciascuna espressione a valle in cui compare il letterale n con la corrispondente espressione del nodo



- Se il nodo da eliminare ha un'espressione con `l` letterali e compare n volte come letterale in altri nodi, la variazione complessiva del costo è pari a:

$$\Delta costo = nl - n - l$$

- Alle volte viene espressa una soglia `s` sulla variazione del costo affinché la trasformazione sia accettata: `eliminate(f, s)`

Eliminate

Trasformazione globale

- Esempio: `eliminate(f, 2)`

```
f = a + b + c
x = fe + ed
y = fd + e
```

Costo = 10



```
Δcosto = 2*3 − 2 − 3 = 1
1<2 → ACCETTATA!
```



```
x = (a + b + c)e + ed
y = (a + b + c)d + e
```

Costo = 11

Δcosto in funzione di n e l

	n						
l	1	2	3	4	5	6	7
1	**-1**	**-1**	**-1**	**-1**	**-1**	**-1**	**-1**
2	**-1**	0	1	2	3	4	5
3	**-1**	1	3	5	7	9	11
4	**-1**	2	5	8	11	14	17
5	**-1**	3	7	11	15	19	23
6	**-1**	4	9	14	19	24	29
7	**-1**	5	11	17	23	29	35
8	**-1**	6	13	20	27	34	41
9	**-1**	7	15	23	31	39	47

Sweep

Trasformazione globale

- `sweep()` è un caso particolare di `eliminate` in cui vengono eliminati tutti i nodi con **un solo letterale** o contenenti una costante
- La variazione di costo è sempre `-1` per ogni nodo eliminato (contenente un letterale)

- Esempio: `sweep()`

```
k = a'
j = k + b
h = bk'
```

Costo = 5



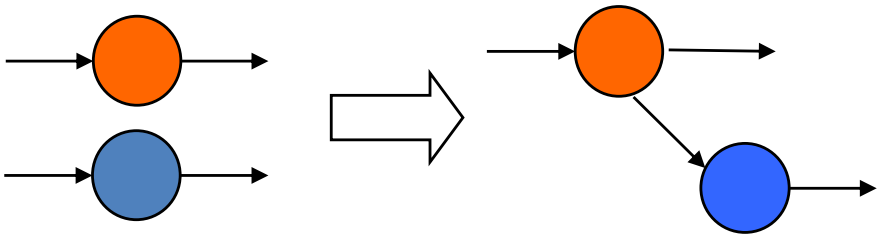
```
j = a' + b
h = ba''
```

Costo = 4

Substitute

Trasformazione globale

- `substitute(f, g)` semplifica l'espressione del nodo `f` **riscrivendola in funzione** di `g` (se possibile)



- Specificando un singolo nodo `g` come parametro, la trasformazione può esser invocata su tutti i nodi a valle della rete cercando di effettuare la riscrittura dell'espressione in funzione di `g`

Substitute

Trasformazione globale

- Se il nodo f è **già fattorizzato** e l'espressione di g vi compare come fattore, la sostituzione avviene semplicemente sostituendo la sottoespressione con il letterale g

- Esempio: substitute(f, g)

```
f = cd(a + b') + a'b(c' + d')
g = a + b'
```

Costo = 10



```
f = cdg + a'b(c' + d')
g = a + b'
```

Costo = 9

Substitute

Trasformazione globale

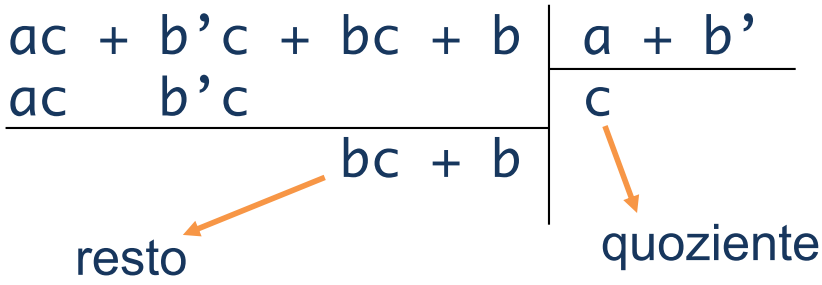
- Se il nodo f è composto da un'espressione su due livelli, si procede eseguendo la divisione algebrica di f rispetto a g:

$$f = f_q\, g + f_r$$

- **Modello algebrico:** le espressioni booleane sono considerate come polinomi di primo grado nelle variabili naturali o complementate e con coefficienti unitari
- Esempio: substitute(f, g)

f = ac + b'c + bc + b
g = a + b'

Costo = 9



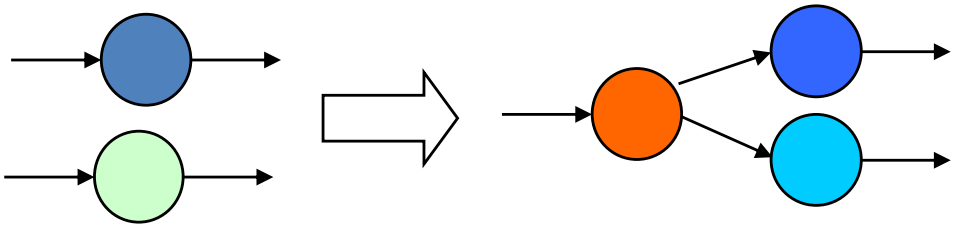
f = cg + bc + b
g = a + b'

Costo = 7

Extract

Trasformazione globale

- `extract(f, g)` estrae dai due nodi `f` e `g` una sottoespressione comune (fattore o divisore comune) generando un nuovo nodo



- Se il fattore o divisore estratto è un'espressione uguale a quella di un altro nodo già presente a monte nella rete, se specificato, si riutilizza tale nodo
- Esempio: `k = extract(f, g)`

```
f = cd(a + b') + a'b
g = a + b' + c
```

Costo = 9



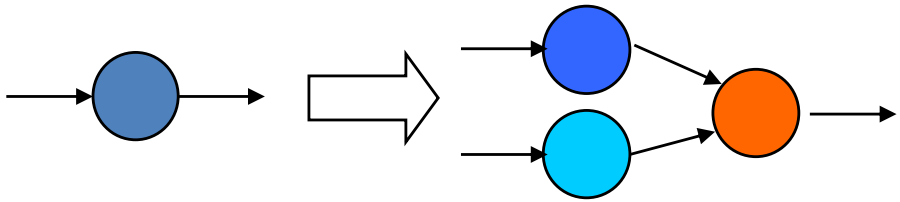
```
k = a + b'
f = cdk + a'b
g = k + c
```

Costo = 9

Decompose

Trasformazione globale

- decompose(f, a, b, …) è una trasformazione che estrae dal nodo f 2^k nuovi nodi mediante l'applicazione del teorema di espansione di Shannon su k variabili (a, b, …)



- Se una delle espressioni estratte è uguale a quella di un altro nodo già presente a monte nella rete, se specificato, si riutilizza tale nodo
- Esempio: $[f_a, f_{a'}]$ = decompose(f, a)

$f = a'b + c + ad$

Costo = 5

$f = af_a + a'f_{a'}$
$f_a = 0b + c + a1 = c + d$
$f_{a'} = 1b + c + a0 = b + c$

Costo = 8